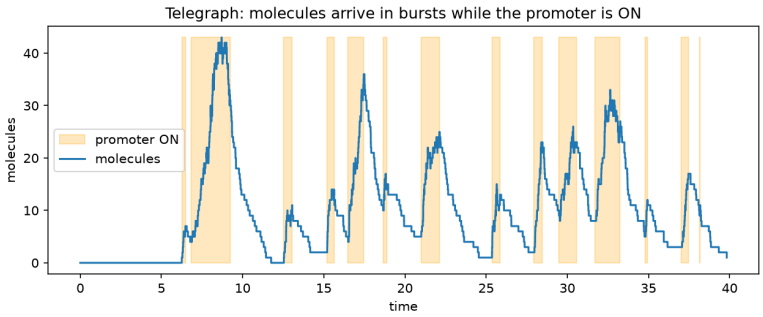
THE UNIVERSITY OF MELBOURNE

Final Report

The Art of Scientific Computing: Bayesian Inference of Transcriptional Bursting from Single-Cell Fluorescence Movies

Author:
Jacob Jurek

Subject Handbook code:
COMP-90072



Faculty of Science
The University of Melbourne

October 23, 2026

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

Abstract

Two identical cells exposed to identical environmental conditions might be expected to respond identically, but the promoter controlling a gene switches on and off at random, so a cell may make a lot of protein in a burst or lie dormant instead. In HIV, these bursts can be the difference between a cell actively making virus and going latent, and because antivirals can only block replicating virus, latent cells survive treatment and can burst again once it stops. The dataset is comprised of movies of cells that have had a GFP instruction attached to their promoter region, which allows me to track frame-by-frame luminosity. Only the brightness of each cell can be observed, and it must be used to work out the rates behind it: how often the promoter turns on, how long it stays on, and how much protein it produces while on. Since the data is only given on a pixel-by-pixel basis, each cell must be segmented and tracked and then given some value to represent its luminosity at each timestep. Cellpose is used along with an implementation of the Hungarian algorithm to identify and track cells. This yielded 479 cells present in all 72 frames from hour 1 to 19 in one well. I compared the simulator against two models before proceeding, and I similarly validated the ABC on synthetic data with known rates. While it met the coverage requirement, the intervals were too wide to be useful, implying that the estimates themselves, while accurate, were uncertain. Compared with an exact-likelihood benchmark, this suggests the information was in the data and rejection ABC lost it; regression adjustment recovered it, and coverage tests confirmed the intervals stayed calibrated. On the real well, a large-number telegraph model that gives each cell its own brightness finds a burst frequency of 0.48 [0.08, 2.3] per hour, in line with the Dar lab's own estimate of 0.42, and an extrinsic spread $\eta = 0.59$ [0.53, 0.66], so most of the variation is between cells. The model reproduces the population statistics but not individual cells: real cells drift slowly where the model expects them to revert, so simply assuming each cell stays where it was makes better forecasts.

Contents

Abstract **1**

1 Introduction **3**
1.1 Noise in gene expression . . . 3
1.2 Aims and outcomes . . . 3

2 Methods **4**
2.1 Data: from microscopy movie to single-cell traces . . . 4
2.2 Stochastic models of gene expression . . . 5
2.3 A telegraph model for fluorescence . . . 5
2.4 Bayesian inference . . . 6
2.5 Software design and computation . . . 6

3 Validation and the Diagnostic Study **7**
3.1 Testing the simulator against exact results . . . 7
3.2 Baseline ABC recovers the rates, but loosely . . . 7
3.3 Diagnosing the wide intervals . . . 8

4 Results: the HIV Latency Circuit **10**
4.1 Calibration on the fluorescence model . . . 10

4.2 Posterior for well K16 . . . 10
4.3 Model checks and held-out tests . . . 10
4.4 Comparison with the published analysis . . . 11

5 Discussion and Conclusions **12**
5.1 What the results say . . . 12
5.2 Limitations and deviations from the plan . . . 12
5.3 What I learned . . . 13
5.4 Future work . . . 13
5.5 Conclusion . . . 13
Code availability . . . 13

Appendix A Code Listings **14**

Appendix B Reproducing the Results **16**

Appendix C Supplementary Results **19**
C.1 Data pipeline . . . 19
C.2 Simulator validation . . . 19
C.3 Further diagnostics on synthetic data . . . 21
C.4 Fluorescence model details . . . 21
C.5 Further results on the HIV data . . . 24

Bibliography **24**

Chapter 1

Introduction

1.1 Noise in gene expression

Two cells with identical DNA, in exactly the same environment, can still end up with drastically different protein levels. Elowitz et al. tested this by putting two copies of the same gene, with identical promoters but different colored fluorescent reporters, into the same cell. The copies still produced different amounts of fluorescence, showing that at least part of the randomness in gene expression is inherent to the cell itself [3]. This randomness arises as a gene's promoter switches on and off at random, producing molecules in bursts rather than steadily. Bursty and steady genes may average the same, but the bursty one has far larger peaks and dips, which can decide whether a treatment works or fails on two otherwise identical cells.

This is clearest in HIV. The virus produces a protein, Tat, that boosts its own expression. If an early random burst produces enough Tat, this feedback takes off and the cell actively makes virus; otherwise the virus goes latent [11]. Antiretroviral drugs only act on actively replicating virus, so latent cells survive treatment and can reappear when the promoter turns back on. Lu et al. [5] screened 1,806 compounds for how they alter this noise, producing the time-lapse dataset used here [6].

Only the brightness of each cell can be observed, and it must be used to work out the rates behind it: how often the promoter turns on, how long it stays on, and how much protein it produces while on. This is an inverse problem. Because the process is inherently random, many different parameter values could explain the observed data, and the model has no tractable likelihood for time-series data. This project therefore uses simulation-based Bayesian inference, specifically approximate Bayesian computation (ABC) [1], which gives a range of plausible values rather than a single estimate, and so also shows how uncertain each rate is.

1.2 Aims and outcomes

The aim of this project was to extract single-cell traces from the public UIUC dataset [6], build a stochastic simulator of the HIV promoter as a telegraph model, and recover its rates with Bayesian methods as a range of plausible values rather than a single estimate. Table 1.1 compares the plan with what was delivered. Overall, the project did what it set out to do, with three deviations (Section 5.2). The Cellpose work [7] was a proof of concept on a single well, since the full screen was too large to process with the available storage and compute. ABC became the main method instead of particle MCMC because it passed the exact-likelihood and coverage checks [9]. And rather than following through with the extensions, the time went into diagnosing and fixing the wide credible intervals that came up. Chapter 2 describes the methods and how the code is organized, Chapter 3 the validation and diagnostic study, Chapter 4 the results, and Chapter 5 the discussion and what I learned.

Plan stage	Planned deliverable	Outcome
Week 1: data	Single-cell traces from raw movies (Cellpose + Hungarian tracking)	Done. 479 cells × 72 frames from one well (K16), consistent with the authors' traces.
Week 2: simulator	Gillespie SSA for birth–death and telegraph models, checked against exact results	Done. Matches the Poisson and Peccoud–Ycart distributions.
Week 3: synthetic inference	ABC recovering known rates with credible intervals	Done. Intervals were wide; a diagnostic study found and fixed the cause (Section 3.3).
Week 4: real data	Posterior for switching rates and burst size of the HIV circuit	Done, with a large-number telegraph model (Ch. 4).
Week 5: extensions	Feedback circuits; DNA strand-displacement link	Partly. Added held-out tests, particle-filter forecasting and comparison with the lab's model; feedback and DNA are discussed only.

Table 1.1: Project plan from the proposal against what was delivered.

Chapter 2

Methods

2.1 Data: from microscopy movie to single-cell traces

The data come from the Dar lab HIV latency screen [5, 6]. Jurkat T cells carry a minimal HIV circuit (LTR-d2GFP-IRES-Tat), in which the HIV promoter drives both a green fluorescent protein (GFP) and Tat, which feeds back positively on the promoter [11]. The GFP decays relatively fast (half-life 2.55 h), so a bright cell indicates a recently active promoter; if it lasted for days, dormant periods would still appear bright. One well (K16) was analyzed: 192 frames 15 min apart, 2044×2048 pixels, one GFP channel. Given each movie is about 1.6 GB and Cellpose takes about 5.5 hours per well on a laptop GPU, processing every well was not feasible in storage or compute, and one well was enough to compare closely with the authors' results.

Cells are segmented in every frame with Cellpose-SAM [7] (Appendix Fig. C.4) and tracked frame to frame by matching centroids with the Hungarian algorithm (SciPy's `linear_sum_assignment`; Appendix Fig. C.2). The tracker assumes a cell cannot move more than 15 μm in one frame and allows cells to disappear for up to 2 frames; tracks shorter than 72 frames are dropped. The mean brightness inside each cell's mask is the measure of how active its promoter is. When a cell divides, one daughter keeps the track and the other starts a new one, as the authors did. This yielded 479 cells present in all 72 frames from hour 1 to 19, the window the authors' published traces cover, with a median CV^2 within 11% of the authors' 59 published traces (0.0042 vs 0.0047; Appendix Fig. C.1).

The background (median of non-cell pixels) falls while the cells get brighter, so the population signal roughly doubles over 18 hours in every well, including the untreated controls. Each frame is therefore rescaled to a flat population mean (Eq. (2.1), Fig. 2.1). This removes the shared rise in brightness and avoids inflating the fitted rates, with the caveat that if some of the rise came from real promoter activity, it was effectively removed from the model; this is tested in Section 4.3. 88% of the variance is between cells rather than over time: each cell holds its own level, and those levels differ far more from cell to cell than any one cell changes over time.

$$\tilde{y}_{c,f} = y_{c,f}\,\frac{\bar{y}}{\bar{y}_f}, \qquad \bar{y}_f = \frac{1}{N_c}\sum_c y_{c,f}, \qquad \bar{y} = \frac{1}{N_f}\sum_f \bar{y}_f \tag{2.1}$$

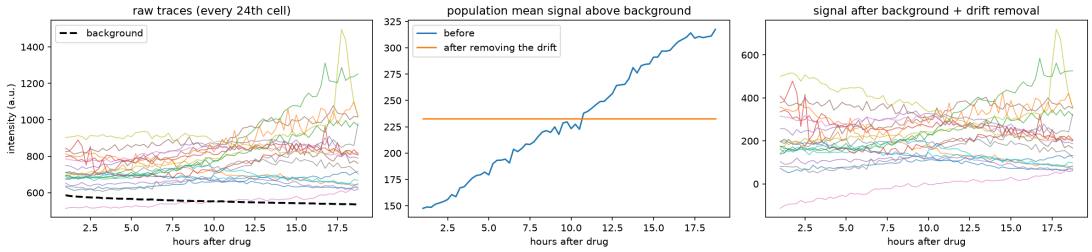


Figure 2.1: Left: raw traces (every 24th cell) with the per-frame background (dashed). Middle: the population mean signal doubles over 18 h; drift removal flattens it. Right: traces after background subtraction and drift removal, the input to inference.

2.2 Stochastic models of gene expression

The telegraph model, Eq. (2.2), is a reaction network [12] and was the obvious choice given the scope of the problem and the work already built around it: the promoter switches ON and OFF at random and makes molecules only while ON. As $k_{on} \to \infty$ it reduces to the birth–death model, whose steady state is Poisson. Simulating birth–death first was a safeguard: if the simulator could not reproduce a Poisson distribution, it could not be trusted on the telegraph model, which is then tested against its exact distribution, Eq. (2.3) [8] (moments in Appendix C.2). Burst size, Eq. (2.4), is a ratio, k_m/k_{off}, not a rate: doubling both leaves the data almost unchanged, so burst size can be well determined even when neither rate is.

$$G_{off} \xrightarrow{k_{on}} G_{on}, \qquad G_{on} \xrightarrow{k_{off}} G_{off}, \qquad G_{on} \xrightarrow{k_m} G_{on} + M, \qquad M \xrightarrow{\gamma} \varnothing \tag{2.2}$$

$$P(n) = \frac{\lambda^n}{n!}\,\frac{\Gamma(a+n)}{\Gamma(a)}\,\frac{\Gamma(a+b)}{\Gamma(a+b+n)}\;{}_1F_1(a+n;\,a+b+n;\,-\lambda), \qquad a = \frac{k_{on}}{\gamma},\; b = \frac{k_{off}}{\gamma},\; \lambda = \frac{k_m}{\gamma} \tag{2.3}$$

$$\text{burst frequency} = k_{on}, \qquad \text{burst size} = \frac{k_m}{k_{off}}, \qquad \text{fraction ON} = \frac{k_{on}}{k_{on}+k_{off}} \tag{2.4}$$

Gillespie's direct method [4] draws exact sample paths: it draws when the next reaction happens, Eq. (2.5), then which reaction fires, with probability a_j/a_0. It is written once, as a generic `gillespie` function that takes a propensity function and a list of state changes (Listing A.1). Both models run through this same function, which is what makes the birth–death safeguard meaningful: with two separate simulators, passing the Poisson test would say nothing about the telegraph one.

$$\tau = -\frac{\ln u_1}{a_0(\mathbf{x})}, \qquad j = \min\Big\{j' : \sum_{i \le j'} a_i(\mathbf{x}) > u_2\, a_0(\mathbf{x})\Big\}, \qquad a_0 = \sum_j a_j, \quad u_1, u_2 \sim U(0,1) \tag{2.5}$$

2.3 A telegraph model for fluorescence

GFP is present at thousands of molecules per cell, so Gillespie would need tens of millions of reactions per simulated movie. In this large-number limit the promoter still switches at random, but between switches the signal relaxes smoothly towards its new level, Eq. (2.6), with time in reporter lifetimes ($\tau_{\mathrm{GFP}} = 2.55\,\mathrm{h}/\ln 2 = 3.68\,\mathrm{h}$, so frames are 0.068 lifetimes apart). The model agrees closely with full Gillespie at large k_m (Table C.2). Real cells also differ in brightness for reasons unrelated to switching, such as size or cell-cycle stage: bright cells stay bright, while in the plain model every cell shares one mean (Appendix Fig. C.3). So each cell gets its own brightness, Eq. (2.7), where η is extrinsic noise in Elowitz's sense [3] and $\eta = 0$ is the plain telegraph model. Readings add Gaussian noise σ. A mixes molecules made with brightness per molecule, so burst size is in arbitrary units. Priors are log-uniform except $\eta \sim U(0,1)$ (Appendix Table C.3). Reported quantities: bursts per hour $= k_{on}/\tau_{\mathrm{GFP}}$, burst size $= A/k_{off}$, $T_{on} = \tau_{\mathrm{GFP}}/k_{off}$, $T_{off} = \tau_{\mathrm{GFP}}/k_{on}$.

$$x(t) = A_i\, s + \big(x(t_0) - A_i\, s\big)\, e^{-(t-t_0)}, \qquad s \in \{0,1\} \tag{2.6}$$

$$A_i = A\,\exp\Big(\eta z_i - \tfrac{\eta^2}{2}\Big), \quad z_i \sim \mathcal{N}(0,1), \qquad y_{i,f} = x_i(t_f) + \varepsilon_{i,f}, \quad \varepsilon_{i,f} \sim \mathcal{N}(0,\sigma^2) \tag{2.7}$$

2.4 Bayesian inference

The posterior $p(\theta \mid y) \propto p(y \mid \theta)\, p(\theta)$ needs a likelihood, and for time series from this model there is no tractable formula for one. Approximate Bayesian computation (ABC) needs only a simulator [12, 1]. A reference table is simulated from the prior, each dataset is reduced to summary statistics, and the draws whose statistics are closest to the data are kept, Eq. (2.8). Here ρ is a Euclidean distance with each statistic scaled by its MAD ($m_k = 1.4826\,\mathrm{MAD}$). ABC therefore targets $p(\theta \mid s(y))$ and loses information whenever the statistics are not sufficient and $\epsilon > 0$, which Section 3.3 investigates. The proposal planned particle MCMC, with ABC as a first, simulator-only step. ABC became the main method because it passed the exact-likelihood and coverage checks, and because one reusable reference table made the whole diagnostic study affordable.

$$p_\epsilon(\theta \mid y) \;\propto\; p(\theta) \int \mathbb{1}\big[\rho\big(s(y'), s(y)\big) \le \epsilon\big]\, p(y' \mid \theta)\, \mathrm{d}y', \qquad \rho(s', s) = \sqrt{\textstyle\sum_k \big((s'_k - s_k)/m_k\big)^2} \tag{2.8}$$

Rejection treats every kept draw as an equally good match, though none match exactly. Regression adjustment [1] weights the kept draws with an Epanechnikov kernel, regresses $\log\theta$ on $(s_i - s_{obs})$, and shifts each draw to where its statistics would match the data, Eq. (2.9) (Listing A.2). In the fluorescence model, η is adjusted on its natural scale and clipped to $[0, 1]$. This assumes the relationship is roughly linear among the kept draws, which the coverage test checks.

$$w_i = 1 - (\rho_i/\epsilon)^2, \qquad \log_{10}\theta_i^* = \log_{10}\theta_i - \big((s_i - s_{obs})/m\big)^\top \hat{\beta} \tag{2.9}$$

One synthetic test only shows that the truth landed inside the interval once. A coverage test [9] uses 300 reference rows in turn as fake data with known θ, runs ABC on the remaining rows, and records $\widehat{C}(\alpha)$, the fraction of α-level intervals that contain the truth, with standard error $\sqrt{\alpha(1-\alpha)/300}$. Calibrated means $\widehat{C}(\alpha) = \alpha$. Width matters too, since returning the prior would have perfect coverage.

A particle filter [12] is used only for held-out forecasting. Each particle is a hypothetical cell with its own parameters, promoter state and signal. At each frame the particles are moved forward with the model, weighted by how well they explain the new reading, $w_t^{(i)} \propto \mathcal{N}(y_t;\, x_t^{(i)},\, \sigma^{(i)2})$, and resampled with systematic resampling (Listing A.4), so the particles that fit the data survive.

2.5 Software design and computation

The analysis is split into five numbered Jupyter notebooks, each of which runs in order and answers a single question; additionally, there is a small package, `stochsim`, and a test suite in `tests/` (layout in Appendix Table B.2). The raw movie becomes traces (01), the simulator is checked against exact results (02), inference is checked on synthetic data with known rates (03, 04), one script, `build_reference_table.py`, builds the reference tables in parallel, and notebook 05 produces the posterior, model checks and forecasts. Code moved into `stochsim` only when it was needed by a second piece of code. The clearest example is the reference-table builder, which runs both on EC2 and on the laptop. It has to use exactly the same simulator and summary statistics as the notebooks: if it used a separate copy and one of the two copies was later changed, the table would no longer match the analysis, and nothing would flag the mismatch. So the simulator is written once, as a single `gillespie` function in `stochsim`, and notebooks 02–05 and the table builder all import it. In the same way, every figure is saved by one shared function under a fixed file name, and Appendix B lists which notebook section produces each figure and table.

The Gillespie function depends on its previous state to determine how long to wait for the next reaction and what that next reaction is. That is to say, the loop cannot be vectorized over time. Instead, vectorization is applied along the independent axes. The fluorescence model updates all 479 cells at once with array masks, touching only the cells whose promoter switches (Listing A.3), and ABC distances are computed over the whole 100,000-row table in one array operation rather than a simple Python loop. The tests time both versions on the same inputs: the distances take 0.32 s as a loop and 0.009 s with NumPy, and the lag autocorrelation of a 400×72 movie 5.3 ms against 0.09 ms. Vectorizing is not always worth it: evaluating the exact likelihood over its whole grid at once gained little (20 ms against 17 ms), because the time is spent inside SciPy's ${}_1F_1$, not in Python overhead.

Independent simulations run in parallel. One script, `build_reference_table.py`, builds both tables (`--model gillespie` or `fluorescence`): drawing the prior up front, then cutting the rows into chunks and running the chunks on every core via `multiprocessing`. The 100,000-row Gillespie table costs about 0.5 CPU-seconds per row, roughly 14 CPU-hours in total. It could have run on the laptop, in an estimated two hours on all eight cores, but it was run on a 64-core AWS EC2 `c7a.16xlarge` instead, where it took 13 minutes and did not need the laptop to stay on. To replicate, simply copy the script and package over, start the job under `nohup` so it survives the SSH session closing, copy the results back and terminate the instance. Changing the algorithm

gained more than changing the hardware. At the same rates the large-number model (Section 2.3) simulates a movie about 20× faster than Gillespie (4.8 ms for 479 cells against 98 ms for 400 cells in the tests), and a real-data table row costs about 12 CPU-ms against 500 for a Gillespie row. The 200,000-row table for the real data only took 4.9 minutes on eight laptop cores, with no EC2 needed.

Three numerical issues came up. The Gamma functions in Eq. (2.3) overflow for large n, so they are summed as logarithms. ${}_1F_1(a+n;\, a+b+n;\, -\lambda)$ loses precision at large λ because its terms alternate in sign and cancel, so Kummer's transformation, ${}_1F_1(a+n;\, a+b+n;\, -\lambda) = e^{-\lambda}\, {}_1F_1(b;\, a+b+n;\, \lambda)$, is used instead, in which every term is positive. Then, after subtracting the largest log-weight, the particle weights get normalized, so they never all underflow to zero. To ensure reproducibility, each chunk of a table is seeded from its own child of NumPy's `SeedSequence` (child 0 draws the prior), so a table depends only on the seed and chunk size, not on the number of workers or the order in which chunks finish. Rebuilding the real-data table with the final script reproduced the saved table exactly, and the Gillespie table built on EC2 and on an M2 Mac agrees to about 10^{-15}, because the two processors can round the last digit differently. Settings, package versions and run time are saved with each table in `run_metadata.json`. Every stage was checked against a known answer before it was trusted, and Chapter 3 presents these checks as the project's tests. The fast, repeatable ones are automated in `tests/test_stochsim.py`: nine tests, run in a few seconds with `python -m pytest tests`, covering the Poisson birth–death limit, the Kummer-transformed distribution against the direct formula, the large-number model against Gillespie, each vectorized function against its loop, regression adjustment on a known linear relationship, and a table that is identical whatever the number of workers.

Task	Simulations	Hardware	Wall time	Note
Cell segmentation (Cellpose-SAM)	192 frames	laptop (Apple GPU)	5.5 h	≈100 s per frame
Baseline ABC table (Gillespie)	10,000	laptop, 1 core	5 min	10 cells each
Diagnostic ABC table (Gillespie)	100,000	EC2, 64 cores	13 min	≈14 CPU-hours
Real-data ABC table (large-number)	200,000	laptop, 8 cores	4.9 min	12 vs 500 CPU-ms/row

Table 2.1: Computational cost of each stage. Gillespie cannot be vectorized in time, so speed came from parallelism across simulations and from the large-number approximation. Segmentation time is GPU compute; the run spanned 9.2 h of wall time because the laptop slept.

Chapter 3

Validation and the Diagnostic Study

3.1 Testing the simulator against exact results

The simulator was validated against two models with well-defined answers before proceeding. For birth–death ($k = 20$, $\gamma = 1$) the simulated steady state has mean 19.78 ± 0.10 against 20 and Fano factor 0.979 against 1. For the telegraph model ($k_{on} = 0.5$, $k_{off} = 1.5$, $k_m = 40$) the promoter is ON 25.1% of the time against 25.0%, the mean is 10.27 ± 0.24 against 10 and the Fano factor 10.99 against 11. The histogram matches both the exact distribution, Eq. (2.3), and direct Beta–Poisson sampling (Fig. 3.1a). At fixed mean, the Fano factor falls from 109 to 1.03 as k_{on} rises from 0.05 to 50, recovering the birth–death limit (Fig. 3.1b). The full table and further figures are in Appendix C.2, and the quick checks from this chapter are also automated in `tests/test_stochsim.py` (Section 2.5).

3.2 Baseline ABC recovers the rates, but loosely

The baseline (`03_abc_baseline.ipynb`) tests ABC on synthetic data with known rates, $k_{on} = 0.5$, $k_{off} = 1.5$, $k_m = 40$: 10 cells observed for 50 frames one molecule lifetime apart, with Normal(0, 3) measurement noise.

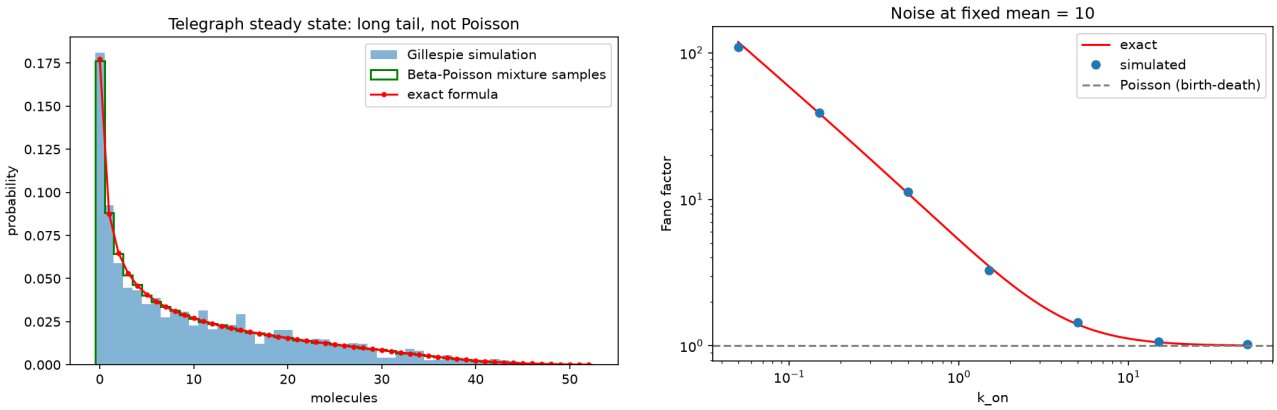


Figure 3.1: (a) Telegraph steady state: Gillespie histogram, exact Peccoud–Ycart distribution and direct Beta–Poisson sampling agree (Fano 11, versus 1 for a Poisson with the same mean). (b) Fano factor against k_{on} at fixed mean: slow switching gives strong bursting noise; fast switching recovers the birth–death limit.

The prior is log-uniform, with $k_{on}, k_{off} \in [0.05, 5]$ and $k_m \in [5, 100]$; 10,000 simulations are reduced to 4 summary statistics and the closest 1% are kept. While for all four test cases the truth remained inside each 95% interval, the intervals were too wide to be considered useful. Rerun with identical settings on the diagnostic table (Table 3.1, Appendix Fig. C.6), they span a factor of 7.6 for k_{on}, 16 for burst size and 64 for k_{off}. Visually, the k_{off} posterior barely shifts from the prior toward the true value.

3.3 Diagnosing the wide intervals

To diagnose the wide intervals caught by this first check, I tested three things. Firstly, Monte Carlo noise, in which case the intervals shrink with more simulations. Then, whether ABC was discarding information, in which case a better method on the same data gives narrower intervals. Finally, whether the data outright lacked the information I planned to infer from the model. A new reference table of 100,000 simulations was drawn from a wider log-uniform prior ($k_{on}, k_{off} \in [0.01, 20]$, $k_m \in [2, 200]$), each simulated under both the baseline design and a more realistic design of 400 cells and 72 frames 15 min apart, based on the actual movie. Statistics were also stored for the first 10, 50 and 100 cells, so the effect of experiment size could be measured without re-simulating.

Going from 10 to 400 cells narrows every interval, with most of the gain reached by 50–100 cells: burst size goes from ×16 to ×3.8, but k_{off} only to ×18.5. Extending the summary statistics from 4 to 11 barely helps plain rejection, because all 11 must match at once, so the closest draws are worse matches on each one. A larger table helps somewhat: with 200 draws kept and all 11 statistics, going from 10,000 to 100,000 simulations narrows k_{off} from ×28 to ×9 (Appendix Fig. C.9; a different setting from the 500 draws kept in Table 3.1). That is to say, Monte Carlo noise is present, but it is not the main contributor.

With regression adjustment on the 11 statistics, the widths fall to ×1.4 for k_{on}, ×1.8 for k_{off} and ×1.4 for k_m and burst size (Table 3.1; posteriors after each change in Appendix Fig. C.7, and a worked example for one rate in Appendix Fig. C.8). The extra statistics only pay off once the regression can use them: rejection needs every statistic to match at once, whereas the regression uses each one to correct the kept draws.

A narrow interval doesn't, however, show that I had found the cause of the problem. For snapshot data, with just one reading per cell, an exact likelihood exists: the telegraph distribution tested in Section 3.1 plus the measurement noise, Eq. (3.1). Evaluated on a 40^3 grid with 400 snapshots, it gives k_{off} a width of ×2.7, against ×18.5 for rejection ABC on the full time series (Fig. 3.2). The information about k_{off} is therefore in the data, and rejection ABC was somehow losing it. Regression ABC (×1.8) can be narrower than the exact snapshot posterior because it also uses the autocorrelations of the time series, which snapshots do not have.

$$p(\mathbf{y} \mid \theta) = \prod_{c=1}^{400} \sum_{n=0}^{\infty} P_{\text{tel}}(n \mid \theta)\, \mathcal{N}(y_c;\, n,\, 3^2) \tag{3.1}$$

The coverage test (Section 2.4) reuses 300 rows of the existing table as fake data, so it needed no new simulations (Fig. 3.3). No method is overconfident. Regression adjustment lies closest to the diagonal, while rejection is too cautious: its 50% intervals contain the truth 66–72% of the time. The main test case is also an easy one. Averaged over the prior, the 95% widths for k_{on}, k_{off} and k_m are ×5.2, ×14.6 and ×1.9 with regression adjustment, against ×13, ×62 and ×3.4 for rejection. Neither regression adjustment nor coverage testing was in the proposal; both were learned for this project, from Beaumont et al. [1] and Prangle et al. [9].

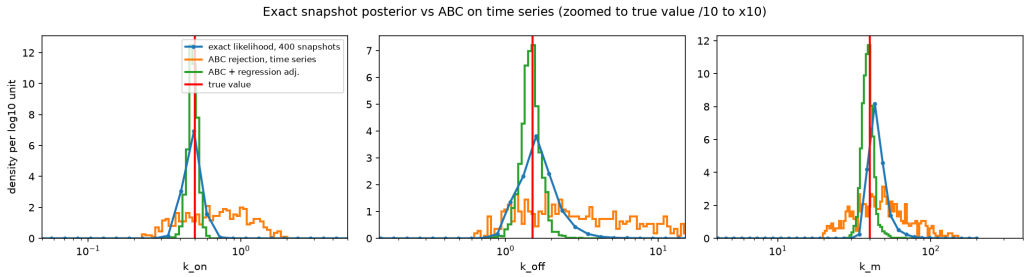


Figure 3.2: Exact snapshot-likelihood posterior (400 cells, one frame each) against rejection and regression-adjusted ABC. The exact posterior is far narrower than rejection ABC, so the information about k_{off} is present in the data.

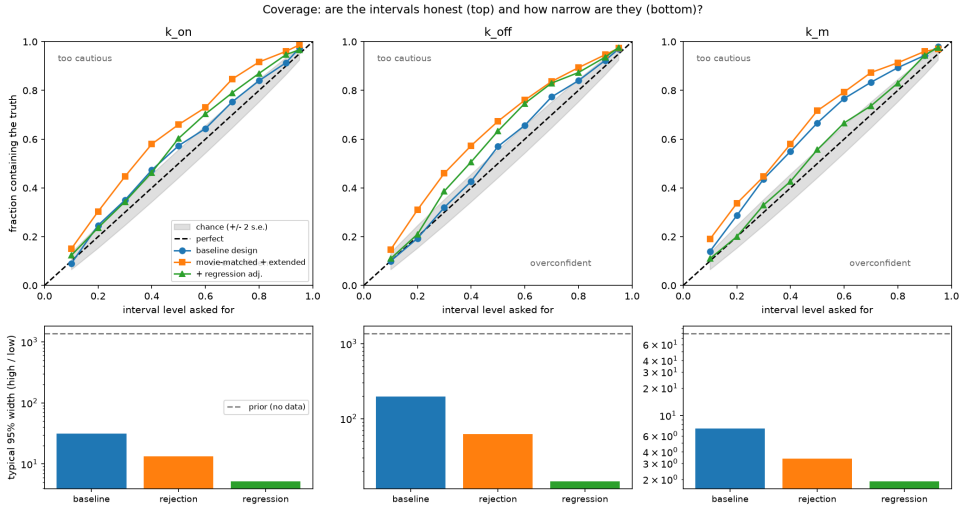


Figure 3.3: Coverage over 300 synthetic datasets. Top: empirical against nominal coverage (diagonal = calibrated, above = too cautious, below = overconfident; grey band ±2 standard errors). Bottom: typical 95% width. Regression adjustment is calibrated and far narrower.

Where switching produces no distinct bursts (the "mostly ON" and "fast switching" cases), k_{off} and burst size stay ×78–98 wide even with the best pipeline, though the truth is still inside. Refitting with only the simulations inside the narrower baseline prior shows how much each method depends on the prior. Rejection's k_{off} width moves from ×18.5 to ×4.1, so the prior is doing most of the work, while regression's widths barely move (×1.4/×1.8/×1.4 against ×1.3/×1.4/×1.1 for $k_{on}/k_{off}/k_m$, truth inside both), so the data decide them. Everything in Chapter 4 builds on these results.

Pipeline	k_{on}	k_{off}	k_m	burst size
Baseline (10 cells, 10k sims, 4 stats)	×7.6	×64	×8.8	×16
Movie-matched design, 400 cells, basic stats	×4.5	×18.5	×8.4	×3.8
400 cells, extended stats	×5.4	×18.5	×5.1	×6.8
400 cells, extended stats + regression	×1.4	×1.8	×1.4	×1.4
Exact likelihood, 400 snapshots	×1.5	×2.7	×1.6	–

Table 3.1: 95% credible-interval width (upper/lower bound) for the main test case after each change. The true value is inside every interval.

Chapter 4

Results: the HIV Latency Circuit

4.1 Calibration on the fluorescence model

The model changed from Section 3.3, so coverage must be rerun. On 300 synthetic datasets, the 95% intervals contain the truth 96–97% of the time for all five parameters (Appendix Fig. C.10). Coverage averages over the whole prior, so recovery was also tested at the posterior median for K16: over 20 movies simulated there and refitted, the truth was inside the 95% interval 20 times for k_{on}, k_{off}, A and σ, and 19 times for η. The intervals there are as wide as on the real data (×29 for k_{on}, ×178 for k_{off}), so the wide switching intervals in Section 4.2 seem to be a property of the data at these values, rather than a failure of this particular method.

4.2 Posterior for well K16

The posterior statistics for the K16 well are summarized in Table 4.1 and Fig. 4.1. The extrinsic spread is $\eta = 0.59$ [0.53, 0.66], and $\eta = 0$, the plain telegraph model, is ruled out, consistent with 88% of the variance being between cells (Section 2.1). The switching rates are weakly determined. Burst frequency is about 0.5 per hour, but its interval spans a factor of 29, and burst size spans a factor of 124. The fluctuations caused by switching are small next to the fixed differences in brightness between cells, meaning they carry little information about the rates. Hence, only A, σ and η are well determined.

4.3 Model checks and held-out tests

The posterior predictive check passes: all 13 statistics of the real data lie within 2 standard deviations of 200 movies simulated from the posterior (Appendix Fig. C.11). η and burst frequency are robust to the background level and to which traces are used, but the fraction of time ON depends on how the drift is handled, rising to 0.90 when only hours 1–10 are used and falling to 0.48 without drift removal (Table C.4).

These checks compare the model with the same cells it was fitted to. Fitting on half the cells and predicting the other half works well, with 97% of held-out statistics within ±2 standard deviations (Appendix Fig. C.12). Splitting by time does not. Fitting hours 1–10 and predicting hours 10–19, only 83% fall within range and the

Quantity	Median	95% credible interval
Burst frequency (bursts per hour)	0.48	0.08 – 2.3
Burst size (a.u., A/k_{off})	460	35 – 4,300
Fraction of time ON	0.67	0.32 – 0.97
Mean ON / OFF period (h)	5.0 / 2.1	0.3 – 48 / 0.4 – 12.7
Always-ON brightness A (a.u.)	350	295 – 444
Measurement noise σ (a.u.)	18	6 – 32
Extrinsic spread η	0.59	0.53 – 0.66

Table 4.1: Posterior summary for well K16 (479 cells, regression-adjusted ABC).

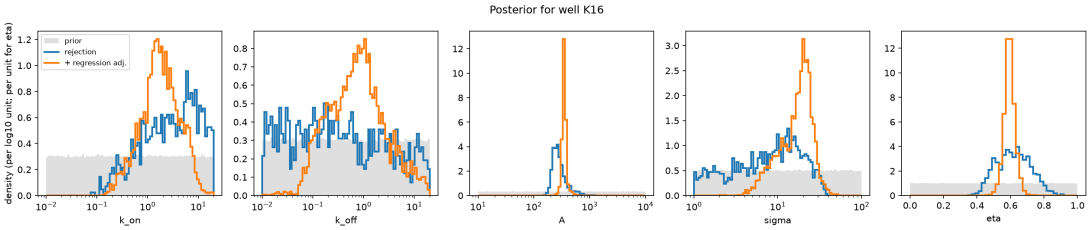


Figure 4.1: Posterior distributions for well K16 from rejection and regression-adjusted ABC, against the prior. A, σ and η are well determined; the switching rates are only loosely constrained.

mean misses at $z = +4.1$. Fitted separately, the two halves give $A = 226$ [193, 271] and 447 [373, 588] a.u., roughly double with no overlap (Appendix Fig. C.13). So even after drift removal, the process is not stationary over the movie.

The most direct test is to forecast each cell's own trace. A bootstrap particle filter with 2,000 particles per cell conditions on hours 1–10 and forecasts hours 10–19. I compared it with two simple forecasters: persistence, where the cell stays where it was at hour 10, and the cell's own average. Persistence wins (Table 4.2, Fig. 4.2). Its 95% bands contain 93% of held-out readings against 62% for the model, and its median error is 60 a.u. against 90. A poor forecast could come from the model or from a bug in the filter. To separate the two, the same filter was run on cells simulated from the model itself, where the model is correct by construction. There it is calibrated (53, 82 and 95.5% for nominal 50, 80 and 95%), so the problem is the model, not the code. Real cells keep drifting in the same direction for hours, whereas the model expects them to return to their own level within about one reporter lifetime. The drift was also removed using all 72 frames, which hands the model the population trend in advance. Persistence still wins, and that makes the result stronger.

Forecaster	50% band	80% band	95% band	Median abs. error	95% width
Particle filter (model)	0.19	0.40	0.62	90	152
Persistence	0.45	0.77	0.93	60	295
Cell average	0.16	0.35	0.61	91	177
Particle filter on model-simulated cells	0.53	0.82	0.955	23	–

Table 4.2: Forecast coverage for hours 10–19 (fraction of held-out readings inside each band) and errors in a.u. The bottom row shows the filter is calibrated when the model is correct.

4.4 Comparison with the published analysis

The paper does not report switching rates, so the comparison was done in two steps. First, I checked the code. Applied to the authors' 59 traces, my implementation of the Lu et al. [5] noise metrics reproduces $\langle FL \rangle$ and HF-CV^2 exactly and HF-$\tau_{1/2}$ to within 7% (Appendix Table C.5). The fitted model also reproduces both noise metrics on simulated movies, even though neither was used in the fit (Appendix Fig. C.14). Second, the rates were compared with the lab's own model for untreated cells [2] (Appendix Table C.6). Burst frequency (0.48 against 0.42 per hour) and the mean OFF period (2.1 against 2.37 h) agree. The ON fraction and ON period do not, which is expected. Without an mRNA step, "ON" in this model really means "GFP is being made", and that covers the promoter's ON period plus the ∼2.4 h lifetime of the mRNA it produced (Appendix Fig. C.15).

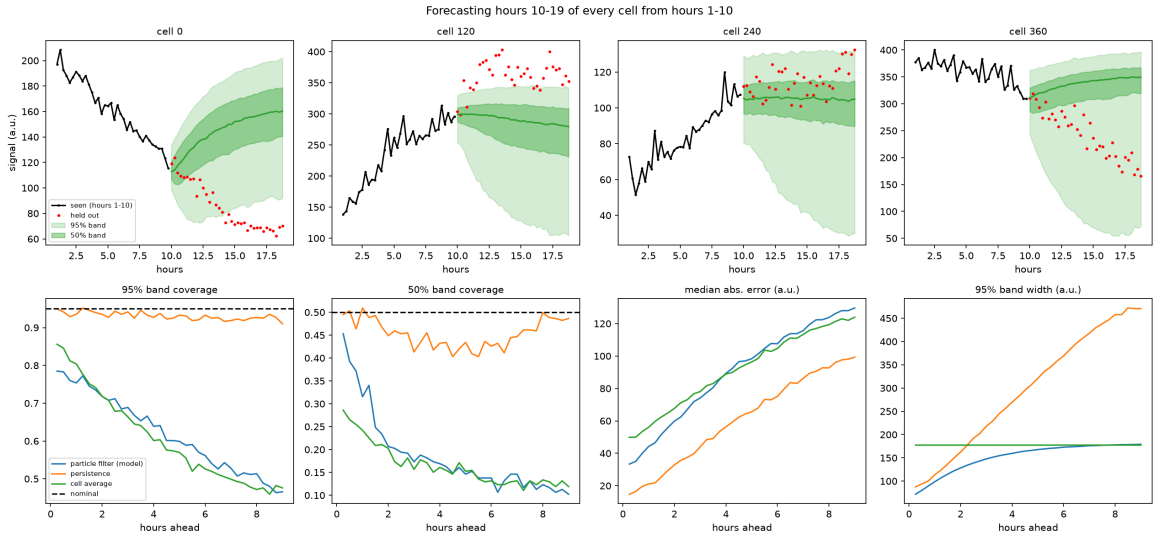


Figure 4.2: Particle-filter forecasts. Top: four cells seen for hours 1–10 (black), held-out readings (red) and the model's 50% and 95% bands. Bottom: coverage, error and band width against forecast horizon for the model, persistence and cell-average forecasters.

Chapter 5

Discussion and Conclusions

5.1 What the results say

Most of the variation in K16 is extrinsic, in the same sense used by Elowitz et al. [3]. Cell-to-cell differences in brightness throughout the movie are much larger than randomness specific to the promoter regions. Hence, the switching rates are poorly constrained. It, again, as seen in Elowitz, would also suggest that a two-color reporter, which further separates intrinsic from extrinsic noise directly, would help more than simply adding cells. That is to say, the model describes the population but not the individual cells it tries to model in the first place. Because it reproduces every summary statistic, the model expects them to revert to some mean within a lifetime, while the actual cells would steadily drift and fluctuate over time. Moreover, when considering that I am explicitly looking at Tat, which is known to have a positive feedback loop, the model ends up ignoring known cell-cycle effects. The missing mRNA step explains why the ON fraction is only an effective quantity, while burst frequency is still comparable with the lab's estimate.

5.2 Limitations and deviations from the plan

Only one well was analyzed, and it is not untreated. K16 received compound NSC 157725 (lab ID D62), which Lu et al. [5] list as an HF-$\tau_{1/2}$ "Decrease" hit (2.35 h against a median of 3.16 h over 34 plate-2 control wells), so the circuit here is perturbed. The background level is uncertain, drift removal shifts the ON fraction, and burst size is only in arbitrary units. Cell division is not modeled either: one daughter keeps the track and the other is dropped.

There were three deviations from the plan. One well was analyzed instead of ∼1,800 conditions, because of the data size and segmentation time. ABC replaced particle MCMC because one reusable reference table, built in parallel, made the calibration tests affordable. In hindsight the forecasting failure backs this up, since

a full trajectory likelihood would have been dominated by the model's misfit to individual cells. The particle filter was still built, and used for forecasting. Finally, the feedback model and the DNA demonstration were not done. The DNA link stays theoretical: any chemical reaction network, including the inferred telegraph model, can in principle be compiled into a DNA strand-displacement circuit [10]. That time went into the diagnostic study, the exact-likelihood benchmark, the large-number model, the held-out tests and reproducing the paper's metrics.

5.3 What I learned

I think the most important takeaway is to treat validation as an initial part of the design rather than a single final step. While this was the initial goal, it was very useful to have a baseline at each point and know when to start asking questions or when it was safe to move on. Every stage of this project had some layer of redundancy. This was most useful with regards to the baseline ABC, where the interval happened to contain the true interval, and I otherwise might have moved on without further investigation. That takeaway is to consider calibration and width together, and check calibration with a coverage test rather than a handful of examples. On the computing side, vectorization became a practical question rather than a rule. Gillespie cannot be vectorized with respect to time, but cells and table rows are independent, and working out which axis could be vectorized decided most of the performance. I was already using EC2 for a MAST90098 project, and running the reference table there was a direct use of the parallel-computing ideas from this one. Even so, switching to the large-number model saved more time than the extra hardware did. Building the filter self-test taught me to separate "the model is wrong" from "the code is wrong" before drawing conclusions, and the forecasting failure said more about the biology than the fit did. I also found it unintuitive that burst size is a ratio, k_m/k_{off}, rather than a rate. That is why it can be well determined when neither rate is.

5.4 Future work

The natural next step is to add an mRNA step and Tat feedback to the model, and to let the rates vary over time instead of removing the drift. Fitting the untreated and control wells from the authors' traces, which are already downloaded, would give a like-for-like comparison with the lab's model. With a validated particle filter already in place, particle MCMC is also within reach.

5.5 Conclusion

The purpose of this project was to see whether I could recover hidden promoter parameters from the HIV latency circuit using single-cell fluorescence movies. Over the course of 7 weeks, it achieved each goal it set out to, excluding the additional extensions, as time was instead spent debugging and addressing issues caught by the earlier layers of redundancy. That is, I derived the traces and luminosity using the pixels from the movie, built and validated a Gillespie simulator, and finally performed and validated inference on the posterior for one well. Through this process, I found that rejection ABC lost information, and found that a regression adjustment could recover it. That well K16 is dominated by extrinsic noise, and the inferred burst parameters agree with the lab's independent estimate. Finally, the model better captures the entire population than individual cells, which are likely a result of the dynamics it leaves out.

Code availability

The code is submitted with this report in the `FinalCode` folder. It needs Python 3.12 with the pinned versions in `requirements.txt`, and the notebooks run in order from `01_extract_traces.ipynb` to `05_hiv_inference.ipynb`. The reference tables are precomputed in `results/` and can be rebuilt with `build_reference_table.py`; the README gives the EC2 steps. Tests run with `python -m pytest tests`. Appendix B maps every figure and table to its code, and the data are available from the Illinois Data Bank [6].

Appendix A

Code Listings

```
def gillespie(start_state, rate_function, constants, changes, t_max, rng):
    time = 0.0
    state = list(start_state)
    times = [time]
    states = [list(state)]

    while time < t_max:
        rates = rate_function(state, constants)
        total_rate = sum(rates)
        if total_rate == 0:
            break

        time = time + rng.expovariate(total_rate)

        pick = rng.random() * total_rate
        reaction = 0
        running_total = rates[0]
        while pick >= running_total:
            reaction = reaction + 1
            running_total = running_total + rates[reaction]

        for j in range(len(state)):
            state[j] = state[j] + changes[reaction][j]

        times.append(time)
        states.append(list(state))

    return np.array(times), np.array(states)

def telegraph_rates(state, constants):
    promoter = state[0]
    molecules = state[1]
    if promoter == 0:
        switch_on, switch_off, make = constants["k_on"], 0.0, 0.0
    else:
        switch_on, switch_off, make = 0.0, constants["k_off"], constants["k_m"]
    destroy = constants["decay_rate"] * molecules
    return [switch_on, switch_off, make, destroy]

TELEGRAPH_CHANGES = [[+1, 0], [-1, 0], [0, +1], [0, -1]]
```

Listing A.1: Generic Gillespie direct method and the telegraph model's propensities (`stochsim/telegraph.py`). A model is only a propensity function plus a list of state changes (Sections 2.2, 3.1).

```
def posterior(stats, params, target, n_keep, regression=False, scale=None, to_fit=np.log10, from_fit=None):
    scale = mad_scale(stats) if scale is None else scale
    rows, distances = closest_rows(stats, target, scale, n_keep)
    if not regression:
        return params[rows], None

    weights = 1 - (distances / distances[-1]) ** 2
    x = (stats[rows] - target) / scale
    design_matrix = np.column_stack([np.ones(len(rows)), x])
    y = to_fit(params[rows])
    root_w = np.sqrt(weights)
    coefficients = np.linalg.lstsq(design_matrix * root_w[:, None], y * root_w[:, None], rcond=None)[0]
    adjusted = y - x @ coefficients[1:]
```

```
    return (10**adjusted if from_fit is None else from_fit(adjusted)), weights
```

Listing A.2: Rejection and regression-adjusted ABC, Eqs. (2.8)–(2.9) (`stochsim/abc.py`, used by notebooks 04 and 05; results in Sections 3.3 and 4.2).

```
def simulate_cells(k_on, k_off, A, sigma, eta, n_cells, np_rng, n_frames=N_FRAMES, frame_spacing=FRAME_SPACING):
    p_on = k_on / (k_on + k_off)
    A = A * np.exp(eta * np_rng.normal(size=n_cells) - eta**2 / 2)
    state = (np_rng.random(n_cells) < p_on).astype(float)
    x = A * p_on
    t = np.zeros(n_cells)
    next_switch = np_rng.exponential(1.0 / np.where(state == 1, k_off, k_on))

    frame_times = BURN_IN + frame_spacing * np.arange(n_frames)
    readings = np.empty((n_cells, n_frames))
    for f, frame_time in enumerate(frame_times):
        switching = next_switch <= frame_time
        while switching.any():
            dt = next_switch[switching] - t[switching]
            target = A[switching] * state[switching]
            x[switching] = target + (x[switching] - target) * np.exp(-dt)
            t[switching] = next_switch[switching]
            state[switching] = 1 - state[switching]
            rates = np.where(state[switching] == 1, k_off, k_on)
            next_switch[switching] = t[switching] + np_rng.exponential(1.0 / rates)
            switching = next_switch <= frame_time
        dt = frame_time - t
        target = A * state
        x = target + (x - target) * np.exp(-dt)
        t[:] = frame_time
        readings[:, f] = x

    return readings + np_rng.normal(0.0, sigma, size=readings.shape)
```

Listing A.3: Large-number telegraph model with extrinsic noise, Eqs. (2.6)–(2.7). All cells are simulated at once; only cells whose promoter switches before the next frame are updated in the inner loop. Time cannot be vectorized, but cells can (Section 2.5) (`stochsim/fluorescence.py`).

```
def systematic_resample(weights, rng):
    positions = (rng.random() + np.arange(len(weights))) / len(weights)
    return np.minimum(np.searchsorted(np.cumsum(weights), positions), len(weights) - 1)


...
for c in range(data.shape[0]):
    A_cell = A0 * np.exp(eta0 * rng.normal(size=n_particles) - eta0**2 / 2)
    state = state_start.copy()
    x = unit_x_start * A_cell
    k_on, k_off, sigma = k_on0.copy(), k_off0.copy(), sigma0.copy()

    for t in range(n_seen):
        if t > 0:
            state, x = fluorescence.propagate(state, x, A_cell, k_on, k_off, dt, rng)
        log_w = -0.5 * ((data[c, t] - x) / sigma) ** 2 - np.log(sigma)
        w = np.exp(log_w - log_w.max())
        keep = systematic_resample(w / w.sum(), rng)
        state, x, A_cell = state[keep], x[keep], A_cell[keep]
        k_on, k_off, sigma = k_on[keep], k_off[keep], sigma[keep]

    for h in range(n_future):
        state, x = fluorescence.propagate(state, x, A_cell, k_on, k_off, dt, rng)
        bands[c, h] = np.percentile(x + sigma * rng.normal(size=n_particles), band_quantiles)
```

Listing A.4: Bootstrap particle filter with systematic resampling (Section 2.4): filter hours 1–10 of each cell, then forecast hours 10–19; results in Section 4.3 (`05_hiv_inference.ipynb`, forecasting; abridged).

Appendix B

Reproducing the Results

Report item	Notebook / script	Section	Output file
Fig. C.4	`01_extract_traces.ipynb`	4. Segment every frame with Cellpose	`data_segmentation`
Fig. C.2	`01_extract_traces.ipynb`	5. Track cells across frames	`data_tracking`
Fig. C.1	`01_extract_traces.ipynb`	7. Check against the authors' published traces	`data_vs_published`
Fig. 2.1	`05_hiv_inference.ipynb`	Background	`data_background_drift`
Table C.1, Fig. C.5	`02_simulator.ipynb`	Model 1: birth–death; Taking steady-state snapshots; Model 2: telegraph; Exact results for the telegraph model	`sim_birthdeath_poisson`, `supp_birthdeath_trajectory`
Fig. 3.1	`02_simulator.ipynb`	Exact results for the telegraph model; How promoter switching controls the noise	`sim_telegraph_exact`, `sim_fano_vs_kon`
Fig. C.3	`02_simulator.ipynb`	What the published control wells look like	`supp_sim_vs_real`
Baseline ABC (Section 3.2)	`03_abc_baseline.ipynb`	Did it find the true rates?; More than one set of true rates	printed
Fig. C.6	`04_abc_diagnostics.ipynb`	Baseline rejection ABC	`abc_baseline_posterior`
Figs. C.8, C.7	`04_abc_diagnostics.ipynb`	Summary statistics and regression adjustment	`abc_regression_example`, `abc_posteriors_each_change`
Fig. C.9	`04_abc_diagnostics.ipynb`	Reference-table size	`supp_abc_table_size`
Fig. 3.2	`04_abc_diagnostics.ipynb`	Exact-likelihood benchmark	`abc_exact_likelihood`
Fig. 3.3	`04_abc_diagnostics.ipynb`	Coverage test	`abc_coverage`
Prior sensitivity	`04_abc_diagnostics.ipynb`	Prior sensitivity	printed
Table 3.1	`04_abc_diagnostics.ipynb`	Interval widths after each change; Exact-likelihood benchmark (last row)	printed
Reference table (100k sims)	`build_reference_table.py --model gillespie` (EC2)	–	`results/diagnostics/`
Table C.2	`05_hiv_inference.ipynb`	Model: telegraph model for large molecule numbers	printed
Reference table (200k sims)	`build_reference_table.py --model fluorescence`	–	`results/hiv/`
Table 2.1	`results/*/run_metadata.json`; `01` (section 4 notes); `03` (Prior and simulations); `tests/`	–	printed
Fig. C.10	`05_hiv_inference.ipynb`	Coverage test on the new model; Recovery at the posterior median	`hiv_coverage`
Table 4.1, Fig. 4.1	`05_hiv_inference.ipynb`	Posterior for well K16	`hiv_posterior`
Fig. C.11	`05_hiv_inference.ipynb`	Posterior predictive check	`hiv_ppc`
Table C.4	`05_hiv_inference.ipynb`	Sensitivity analysis	printed
Figs. C.12, C.13	`05_hiv_inference.ipynb`	Cell split; Time split	`supp_hiv_cell_split`, `supp_hiv_time_split`
Table 4.2, Fig. 4.2	`05_hiv_inference.ipynb`	Forecasting with a particle filter (incl. filter self-test)	`hiv_forecast`
Table C.5	`05_hiv_inference.ipynb`	Comparison with the paper (Lu et al., PNAS 2021)	printed
Fig. C.14	`05_hiv_inference.ipynb`	Does the fitted model reproduce the paper's numbers?	`supp_hiv_paper_metrics`
Table C.6, Fig. C.15	`05_hiv_inference.ipynb`	Switching rates vs the lab's model	`hiv_switching_vs_lab`

Table B.1: Where each result in the report is produced. Output files are PNGs in `figures/`; "printed" means the numbers are printed in that notebook section.

File	Responsibility
`01_extract_traces.ipynb`	Segmentation (Cellpose-SAM), Hungarian tracking, intensity extraction, comparison with the published traces
`02_simulator.ipynb`	Gillespie direct method; birth–death and telegraph models tested against exact results
`03_abc_baseline.ipynb`	Rejection ABC on synthetic data with known rates
`04_abc_diagnostics.ipynb`	The diagnostic study: experiment size, statistics, regression adjustment, exact-likelihood benchmark, coverage
`05_hiv_inference.ipynb`	Inference on well K16, model checks, held-out tests, particle-filter forecasting, comparison with the paper
`stochsim/telegraph.py`	Gillespie, telegraph propensities, experiment designs, priors, summary statistics
`stochsim/fluorescence.py`	Large-number telegraph model with extrinsic noise, drift removal, the 13 summary statistics
`stochsim/abc.py`	Rejection ABC, regression adjustment, MAD scaling, weighted credible intervals
`stochsim/figures.py`	Saves every figure to `figures/` under a stable name
`build_reference_table.py`	Builds either reference table in parallel with `multiprocessing`; the Gillespie table was run on AWS EC2
`tests/test_stochsim.py`	Correctness tests against known answers, and timings of each optimization
`results/`	Saved reference tables, each with its `run_metadata.json` (seed, settings, versions, run time)
`README.md`, `requirements.txt`	Setup, run order, EC2 steps and the result-to-code map; pinned package versions (Python 3.12)

Table B.2: Code layout. Notebooks hold the analysis in order; the package holds code shared between the notebooks and the table builder.

Appendix C

Supplementary Results

C.1 Data pipeline

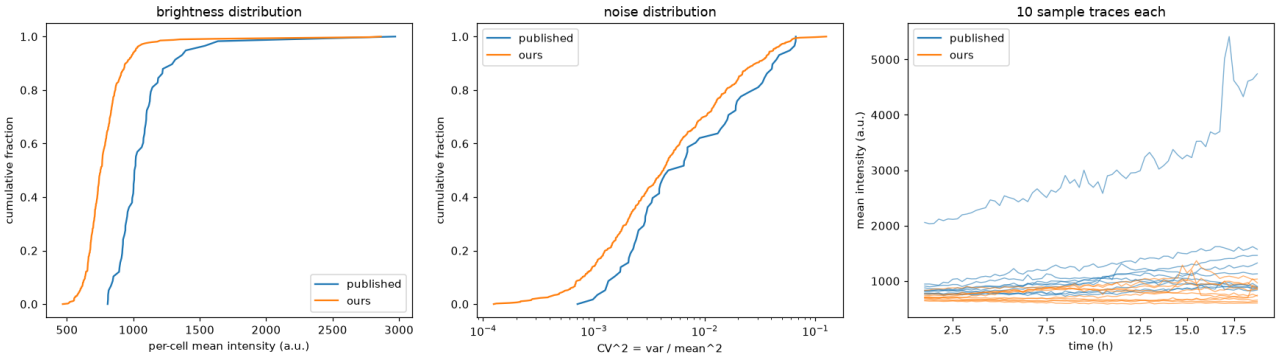


Figure C.1: Traces extracted here (orange, 479 cells) against the authors' published traces for the same well (blue, 59 cells). Left: per-cell mean intensity, lower for the extracted traces because their masks are not eroded towards the bright center. Middle: per-cell CV^2; the distributions agree. Right: ten example traces from each.

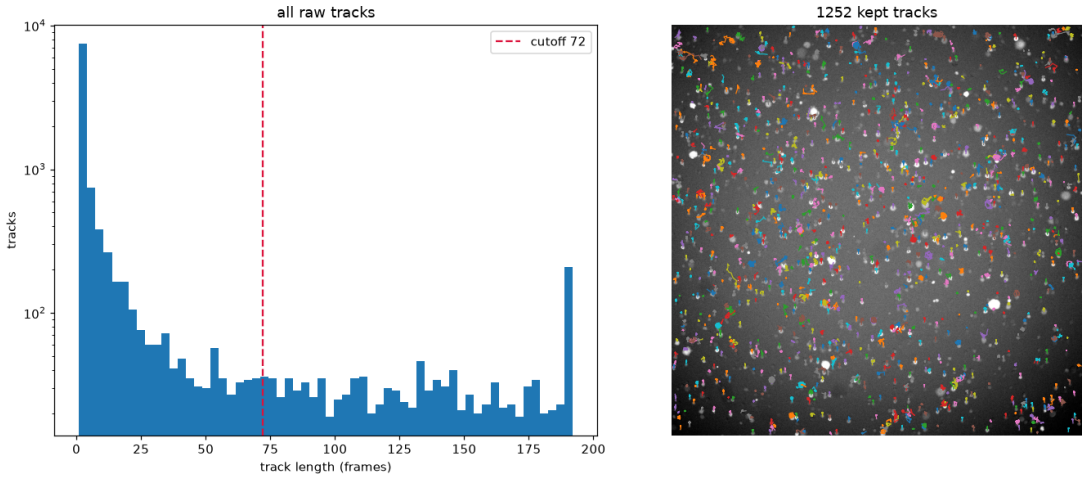


Figure C.2: Tracked cell trajectories. Left: lengths of all 11,252 raw tracks (log scale), with the 72-frame cutoff dashed. Right: the 1,252 tracks of at least 72 frames, drawn over frame 0. Most raw tracks are short fragments, and the kept tracks are local paths with no long jumps.

C.2 Simulator validation

$$\mathbb{E}[n] = \frac{k_m}{\gamma}\,\frac{k_{on}}{k_{on}+k_{off}}, \qquad \text{Fano} = 1 + \frac{k_m\,k_{off}}{(k_{on}+k_{off})(k_{on}+k_{off}+\gamma)} \tag{C.1}$$

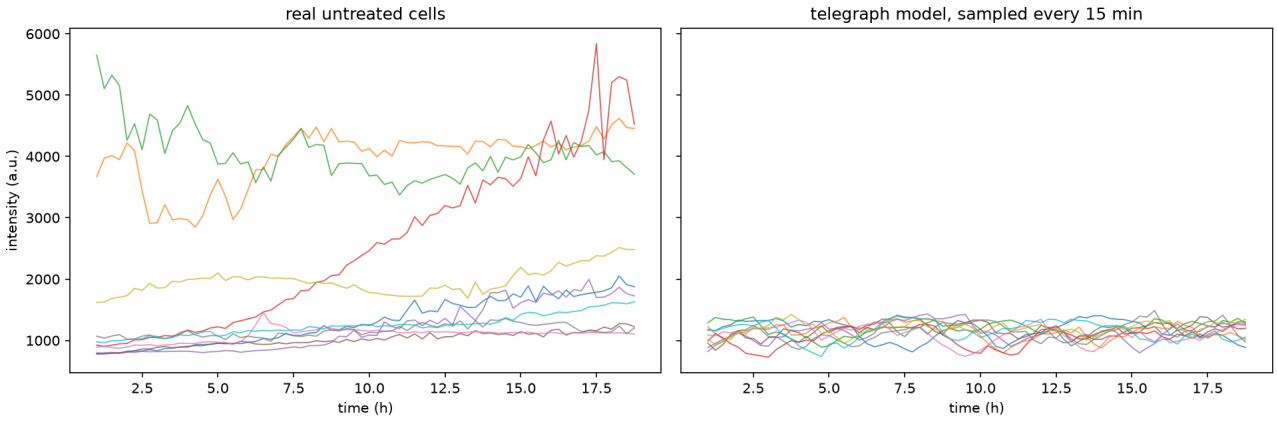


Figure C.3: Real untreated cells (left) against the plain telegraph model sampled every 15 min (right). Real cells differ in level and drift; the model's cells share one mean. This motivated the extrinsic parameter η.

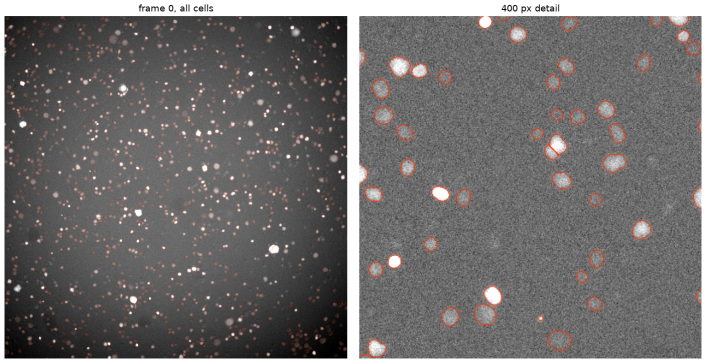


Figure C.4: Cellpose segmentation of well K16 (about 1,000–1,200 cells per frame). Left: frame 0 with all 1,043 mask outlines in red. Right: a 400-pixel detail of the same frame.

Check	Simulated	Exact	Snapshots
Birth–death mean	19.78 ± 0.10	20	1,990
Birth–death Fano factor	0.979	1	1,990
Telegraph fraction of time ON	25.1%	25.0%	–
Telegraph mean	10.27 ± 0.24	10	1,990
Telegraph Fano factor	10.99	11.0	1,990

Table C.1: Simulator validation against exact results. Telegraph parameters: $k_{on} = 0.5$, $k_{off} = 1.5$, $k_m = 40$, $\gamma = 1$.

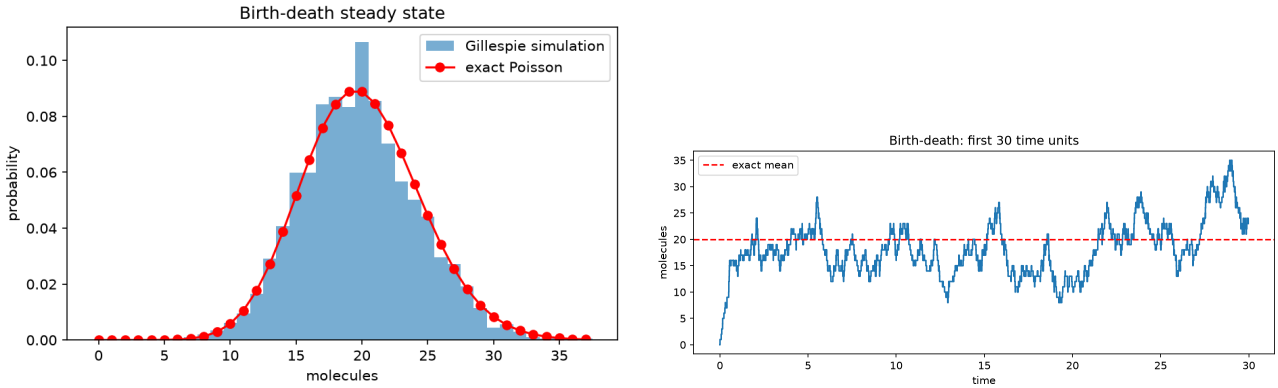


Figure C.5: Birth–death model ($k = 20$, $\gamma = 1$). (a) Steady-state histogram from 1,990 Gillespie snapshots against the exact Poisson(20) distribution. (b) A trajectory relaxing to its steady state, showing the burn-in discarded before snapshots are taken.

C.3 Further diagnostics on synthetic data

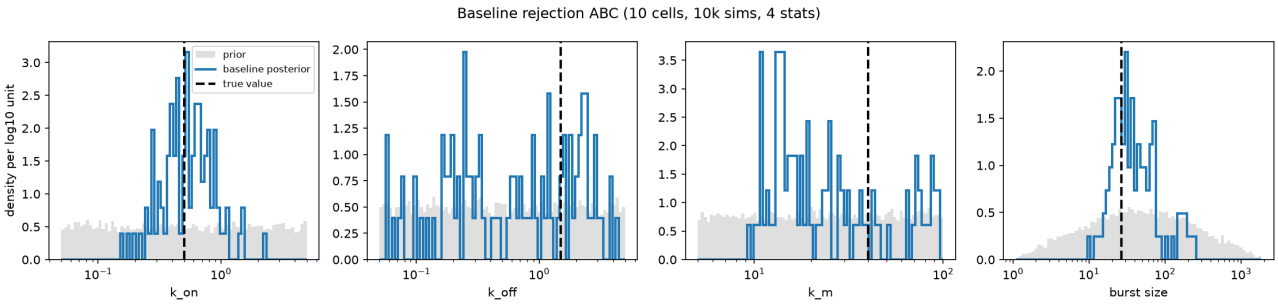


Figure C.6: Baseline rejection ABC (10 cells, 10,000 simulations, 4 statistics): posteriors against the prior, true values dashed. The truth is always inside the 95% interval, but k_{off} is barely narrower than the prior.

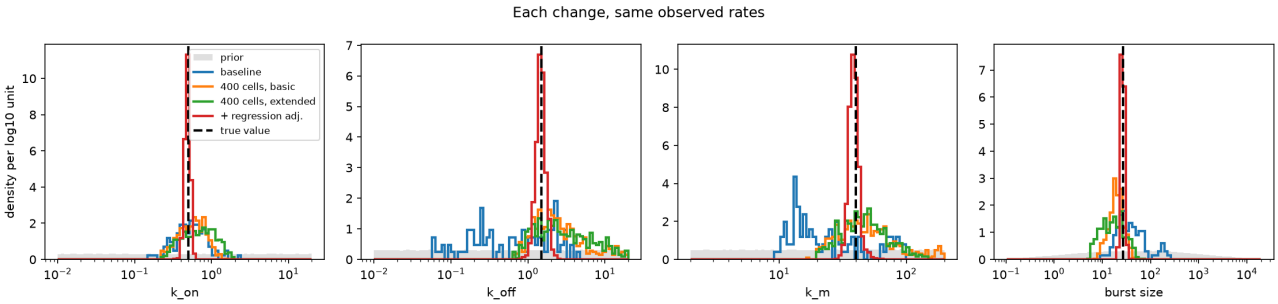


Figure C.7: Posteriors for the same synthetic data after each change: baseline (blue), 400 cells with basic statistics (orange), 400 cells with extended statistics (green), and with regression adjustment (red). Only regression adjustment concentrates the posterior around the true value (dashed).

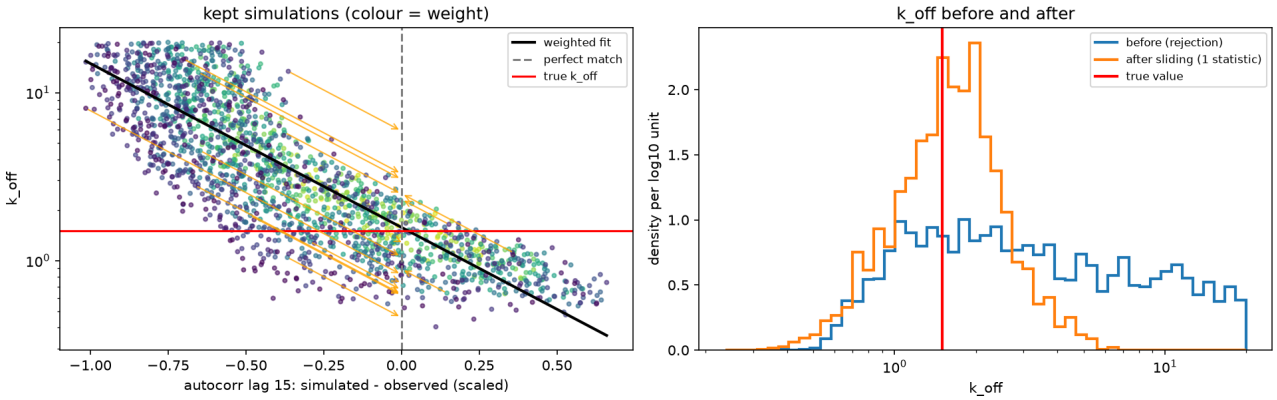


Figure C.8: Regression adjustment for one rate. Left: accepted simulations ($\log k_{off}$ against autocorrelation at lag 15, $r = 0.83$), the weighted fit, and arrows showing samples shifted to the observed value. Right: k_{off} before and after. The real adjustment uses all 11 statistics and all 3 rates.

C.4 Fluorescence model details

$$r(\ell) = \frac{1}{\mathrm{Var}(y)} \frac{1}{N_c(N_f - \ell)} \sum_c \sum_f (y_{c,f} - \bar{y})(y_{c,f+\ell} - \bar{y}), \qquad \mathrm{QS} = \frac{q_{90} + q_{10} - 2\,q_{50}}{q_{90} - q_{10}} \tag{C.2}$$

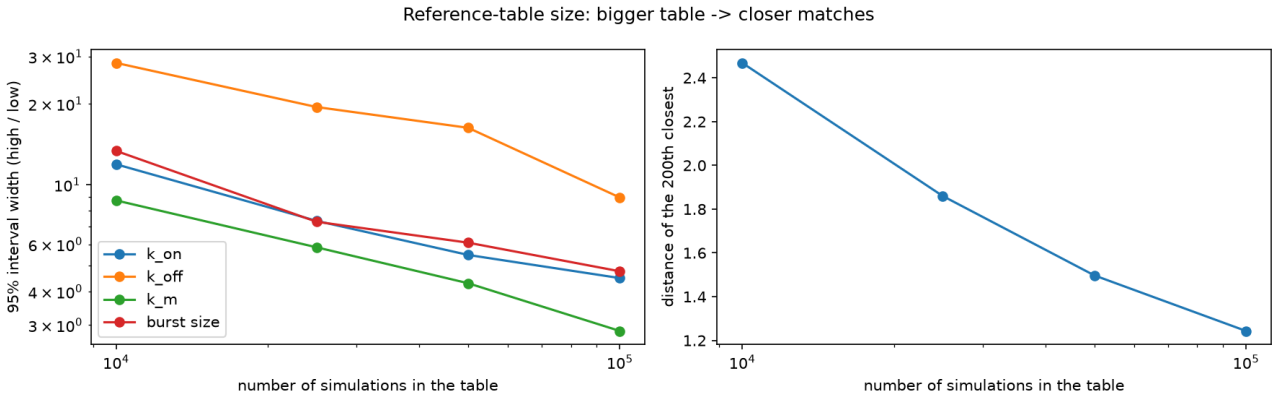


Figure C.9: Effect of reference-table size (10%–100% of 100,000 simulations, same number kept) on rejection ABC: Monte Carlo error contributes, but is not the main cause of the wide intervals.

Statistic	$(0.5, 1.5, 400)$	$(0.1, 1.0, 600)$	$(3.0, 3.0, 300)$
mean	100.0 / 96.8	58.4 / 57.1	154.0 / 150.3
CV^2	1.001 / 1.002	4.49 / 4.58	0.138 / 0.136
autocorr lag 4	0.948 / 0.941	0.974 / 0.962	0.868 / 0.869
autocorr lag 15	0.605 / 0.585	0.729 / 0.687	0.423 / 0.420
autocorr lag 40	0.104 / 0.137	0.173 / 0.215	0.036 / 0.110
q_{90}/mean	2.57 / 2.61	3.82 / 4.02	1.49 / 1.51
between-cell var. fraction	0.471 / 0.460	0.548 / 0.542	0.356 / 0.395
jitter	0.070 / 0.068	0.107 / 0.107	0.045 / 0.041

Table C.2: Gillespie / large-number model for selected statistics at three parameter sets (k_{on}, k_{off}, k_m), 200 cells each.

Parameter	Meaning	Prior
k_{on}, k_{off}	promoter switching rates (per lifetime)	log-uniform $[0.01, 20]$
A	brightness of an always-ON cell (a.u.)	log-uniform $[10, 10^4]$
σ	measurement noise (a.u.)	log-uniform $[1, 100]$
η	extrinsic cell-to-cell spread	uniform $[0, 1]$

Table C.3: Parameters of the fluorescence model and their priors.

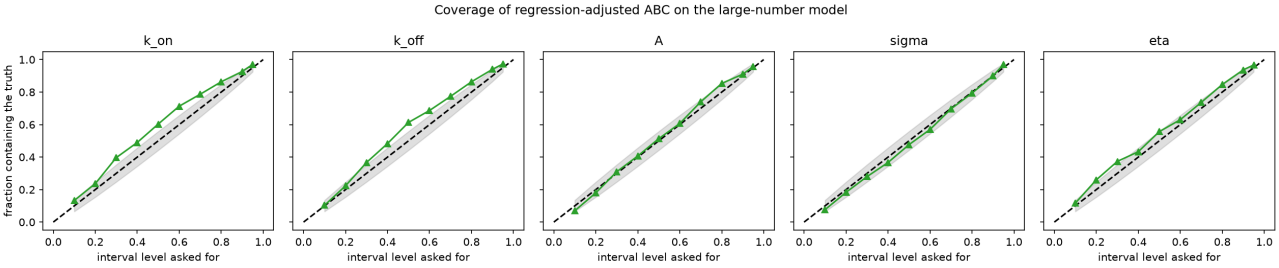


Figure C.10: Coverage of regression-adjusted ABC on the fluorescence model (300 synthetic datasets). All five parameters lie close to the diagonal.

Scenario	Bursts per hour	Burst size (a.u.)	Fraction ON	η
Main analysis	0.48 [0.08, 2.3]	460 [35, 4,300]	0.67 [0.32, 0.97]	0.59 [0.53, 0.66]
Background +25	0.41 [0.06, 2.3]	400 [29, 3,900]	0.61 [0.28, 0.96]	0.65 [0.58, 0.73]
Background −25	0.46 [0.08, 2.0]	610 [47, 5,500]	0.70 [0.36, 0.97]	0.54 [0.48, 0.60]
No drift removal	0.64 [0.11, 3.2]	170 [15, 2,200]	0.48 [0.18, 0.94]	0.57 [0.50, 0.65]
First 36 frames (hours 1–10)	0.53 [0.08, 2.7]	1,300 [99, 10,000]	0.90 [0.70, 0.99]	0.63 [0.57, 0.70]
Authors' 59 traces	1.8 [0.20, 7.0]	930 [68, 9,700]	0.87 [0.55, 0.99]	0.42 [0.29, 0.56]

Table C.4: Sensitivity of the posterior (median [95% interval]) to one analysis choice at a time.

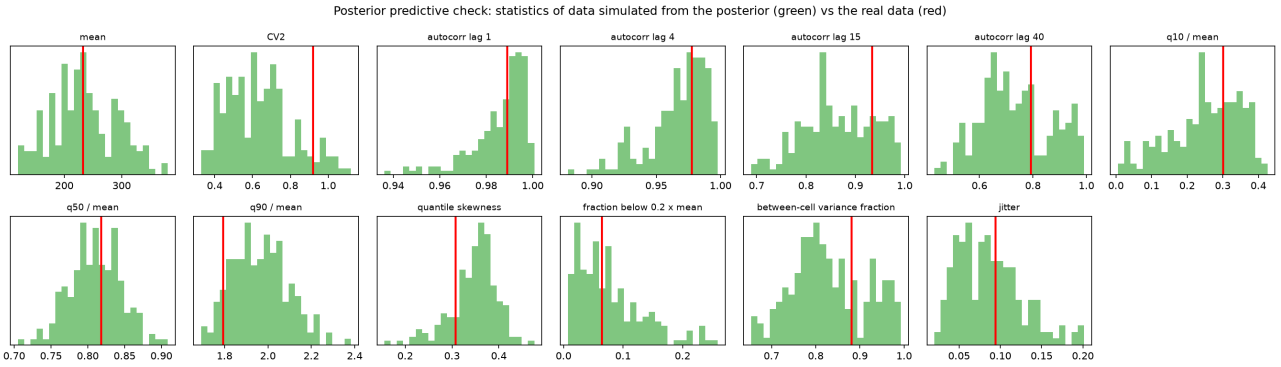


Figure C.11: Posterior predictive check: each summary statistic over 200 movies simulated from the posterior, real value marked. Every statistic is within 2 standard deviations.

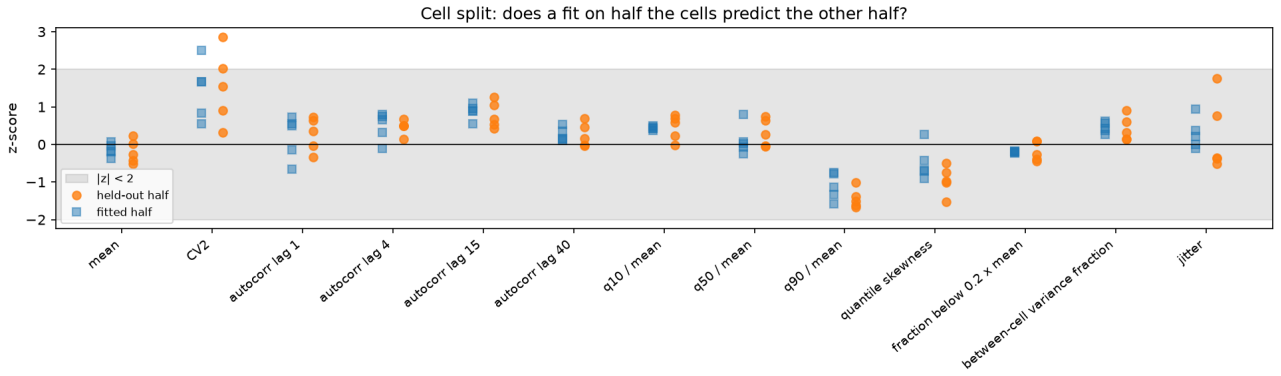


Figure C.12: Cell-split test: z-scores of held-out statistics over five random splits (fit on 240 cells, predict 239). 97% fall within ± 2.

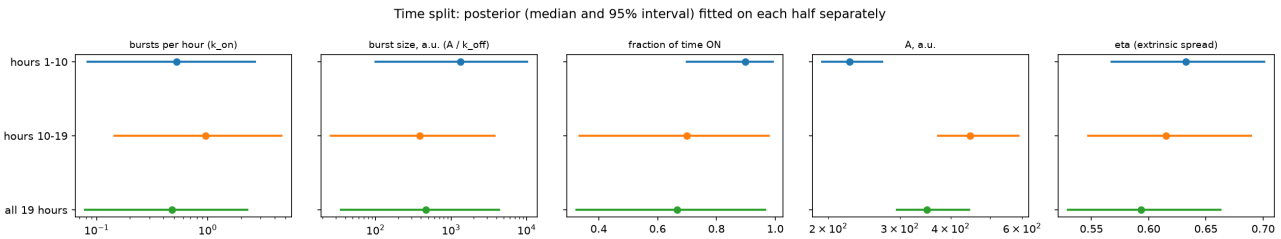


Figure C.13: Time-split test: posteriors fitted separately to hours 1–10 and 10–19. A roughly doubles, so the process is not stationary over the movie.

C.5 Further results on the HIV data

$$\delta_{c,f} = (y_{c,f} - \bar{y}_f) - \frac{1}{N_f}\sum_{f'}(y_{c,f'} - \bar{y}_{f'}), \qquad \langle FL\rangle = \frac{1}{N_f}\sum_{f}\bar{y}_f,$$
$$R(\ell) = \frac{1}{N_c}\sum_{c}\frac{1}{N_f-\ell}\sum_{f=1}^{N_f-\ell}\delta_{c,f}\,\delta_{c,f+\ell}, \qquad \text{HF-CV}^2 = \frac{R(0)}{\langle FL\rangle^2}, \qquad \tau_{1/2} = \min\{\ell : R(\ell) = R(0)/2\} \tag{C.3}$$

	$\langle FL\rangle$	HF-CV^2	HF-$\tau_{1/2}$ (h)
Paper (Dataset S1)	1076.36	0.01749	2.348
Authors' traces, this code	1076.36	0.01749	2.516
Extracted traces (Section 2.1), this code	787.13	0.01123	2.834

Table C.5: Lu et al. noise metrics for well K16.

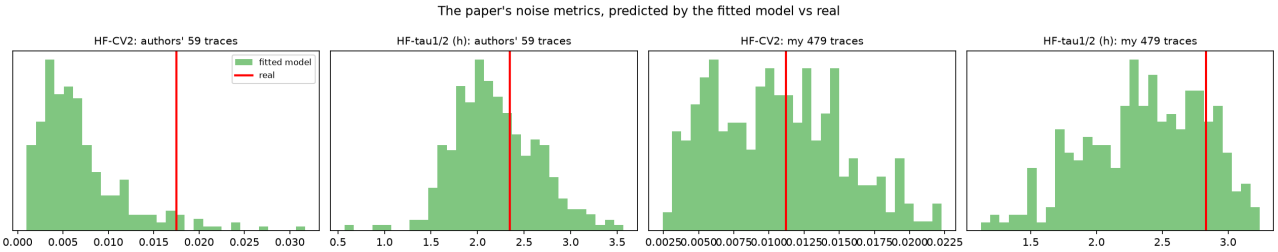


Figure C.14: The paper's HF-CV^2 and HF-$\tau_{1/2}$ computed on movies simulated from the posterior, against the real values. Both lie inside the model's 95% range, although these metrics were not used in the fit.

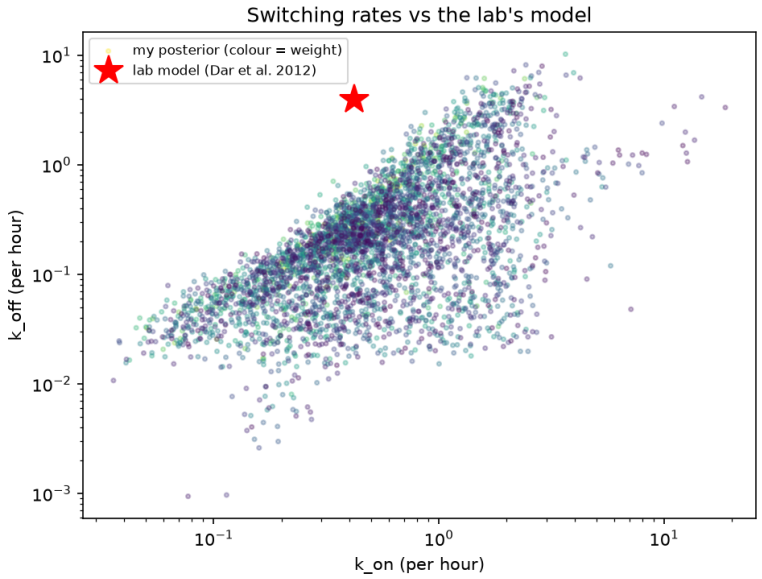


Figure C.15: Posterior over switching rates against the lab's model values. Burst frequency agrees; the ON fraction differs because this model has no mRNA step, so its "ON" state is an effective GFP-production state.

Quantity	Lab model [2]	This work: median [95%]	Consistent?
Bursts per hour	0.42	0.48 [0.08, 2.3]	yes
Mean OFF period (h)	2.37	2.1 [0.4, 12.7]	yes
Fraction ON	0.10	0.67 [0.32, 0.97]	no (effective)
Mean ON period (h)	0.25	5.0 [0.3, 48]	no (effective)

Table C.6: Switching rates compared with the lab's model for untreated cells.

Bibliography

[1] M. A. Beaumont, W. Zhang, and D. J. Balding. Approximate Bayesian computation in population genetics. *Genetics*, 162(4):2025–2035, 2002.

[2] R. D. Dar, B. S. Razooky, A. Singh, T. V. Trimeloni, J. M. McCollum, C. D. Cox, M. L. Simpson, and L. S. Weinberger. Transcriptional burst frequency and burst size are equally modulated across the human genome. *Proc. Natl. Acad. Sci. USA*, 109(43):17454–17459, 2012.

[3] M. B. Elowitz, A. J. Levine, E. D. Siggia, and P. S. Swain. Stochastic gene expression in a single cell. *Science*, 297(5584):1183–1186, 2002.

[4] D. T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977.

[5] Y. Lu, K. Bohn-Wippert, P. J. Pazerunas, J. M. Moy, H. Singh, and R. D. Dar. Screening for gene expression fluctuations reveals latency-promoting agents of HIV. *Proc. Natl. Acad. Sci. USA*, 118(11):e2012191118, 2021.

[6] Y. Lu, K. Bohn-Wippert, P. J. Pazerunas, J. M. Moy, H. Singh, and R. D. Dar. Time-lapse fluorescence microscopy images and gene expression data of single T-cells infected with a minimal HIV feedback circuit under 1,806 drug treatments. University of Illinois Urbana-Champaign (Illinois Data Bank), 2022.

[7] M. Pachitariu, M. Rariden, and C. Stringer. Cellpose-SAM: Superhuman generalization for cellular segmentation. bioRxiv preprint, 2025.

[8] J. Peccoud and B. Ycart. Markovian modeling of gene-product synthesis. *Theoretical Population Biology*, 48(2):222–234, 1995.

[9] D. Prangle, M. G. B. Blum, G. Popovic, and S. A. Sisson. Diagnostic tools for approximate Bayesian computation using the coverage property. *Australian & New Zealand Journal of Statistics*, 56(4):309–329, 2014.

[10] D. Soloveichik, G. Seelig, and E. Winfree. DNA as a universal substrate for chemical kinetics. *Proc. Natl. Acad. Sci. USA*, 107(12):5393–5398, 2010.

[11] L. S. Weinberger, J. C. Burnett, J. E. Toettcher, A. P. Arkin, and D. V. Schaffer. Stochastic gene expression in a lentiviral positive-feedback loop: HIV-1 Tat fluctuations drive phenotypic diversity. *Cell*, 122(2):169–182, 2005.

[12] D. J. Wilkinson. *Stochastic Modelling for Systems Biology*. CRC Press, third edition, 2019.